

Final Project Report

PantryBook — Personal Recipe Manager

Project Title

PantryBook — a responsive, browser-based personal recipe manager.

Project Objective

To design and build a complete, responsive website using the HTML, CSS and JavaScript concepts covered in the internship, and to practise a real Git and GitHub workflow — from local version control through to a live deployment on GitHub Pages.

Project Overview

Family recipes are usually scattered across notebooks, screenshots and chat messages. PantryBook gives them one searchable home. The app ships with a starter set of recipes and lets a user search by name or ingredient, filter by course and difficulty, mark favorites, and add, edit or delete their own recipes through a validated form. All data is saved to the browser's localStorage, so a user's changes persist across page reloads without needing a backend server.

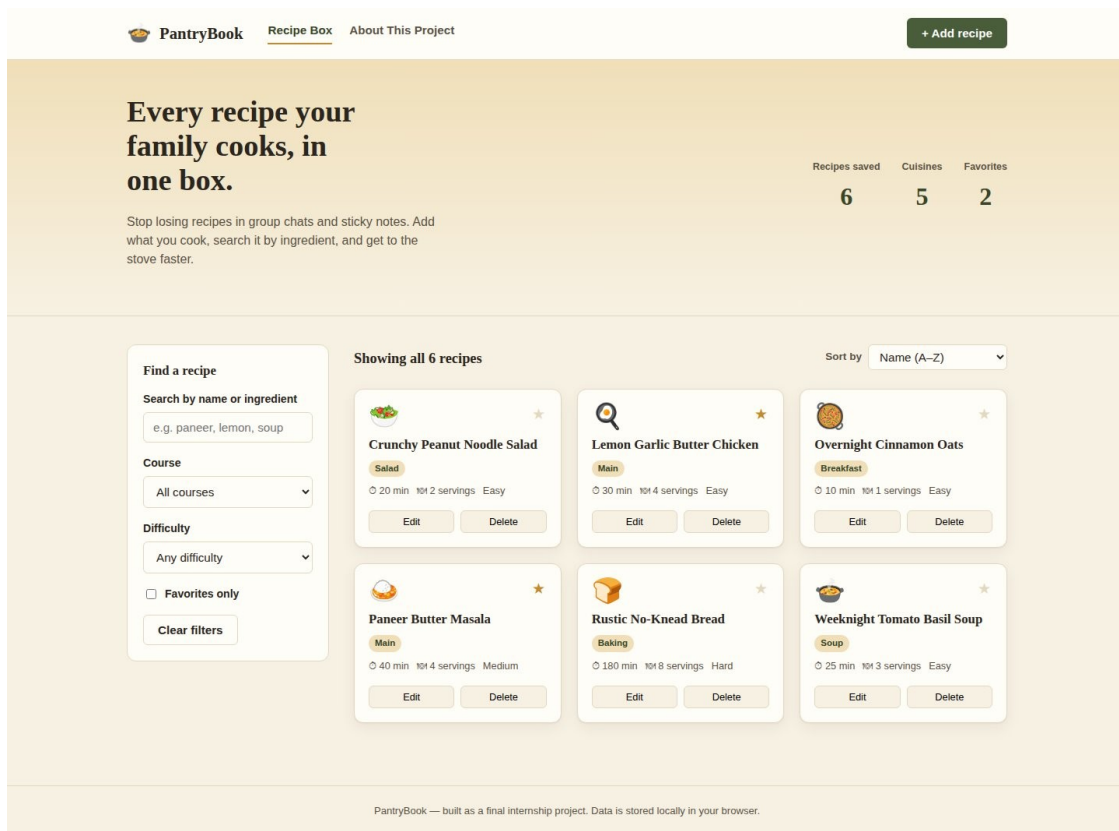
Technologies Used

- HTML5 — semantic structure (header, nav, main, aside, section, article, footer) across two pages.
- CSS3 — the box model, Flexbox (header, filter fields), CSS Grid (recipe card grid), custom properties, and media queries for tablet and mobile breakpoints.
- JavaScript (ES6) — arrays and objects as the data model, DOM rendering, event handling, form validation, and the localStorage API.
- Git & GitHub — local version control with staged, incremental commits, and GitHub Pages for deployment.

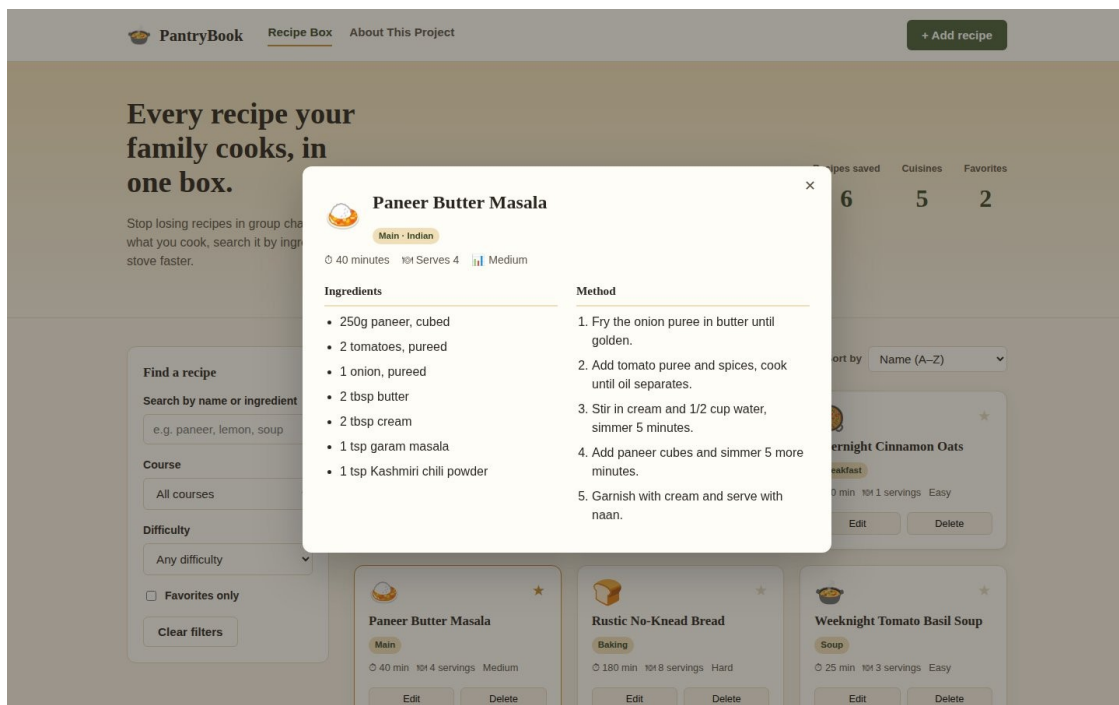
Main Features

- Live search across recipe names and ingredients.
- Filter by course and difficulty, plus a favorites-only toggle.
- Sort by name, cook time, or difficulty.
- Add, edit and delete recipes with inline form validation (required fields, minimum ingredients/steps, numeric ranges).
- One-click favorite toggle reflected instantly in the stats bar.
- Data persistence via localStorage — nothing is lost on refresh.
- Fully responsive layout, verified on desktop, tablet and mobile viewports.

Website Screenshots



Desktop — Recipe Box home, showing live stats, filters and the recipe grid.



Desktop — recipe detail modal with ingredients and method.

PantryBook

Recipe Box

About This Project

+ Add recipe

Every recipe your family cooks, in one box.

Stop losing recipes in group chats and stick what you cook, search it by ingredient, and stove faster.

Find a recipe

Search by name or ingredient

e.g. paneer, lemon, soup

Course

All courses

Difficulty

Any difficulty

☐ Favorites only

Clear filters

Recipes saved

6

Cuisines

5

Favorites

2

Sort by

Name (A-Z)



Overnight Cinnamon Oats

Breakfast

10 min 101 1 servings Easy

EditDelete



Weeknight Tomato Basil Soup

Soup

25 min 101 3 servings Easy

EditDelete

×

Add a recipe

Recipe name *

Course *

Breakfast, Main, Dessert...

Cuisine *

Indian, Italian...

Cook time (minutes) *

Servings *

Difficulty *

Easy

Ingredients * (one per line)

2 cups rice
1 onion, sliced
1 tsp cumin

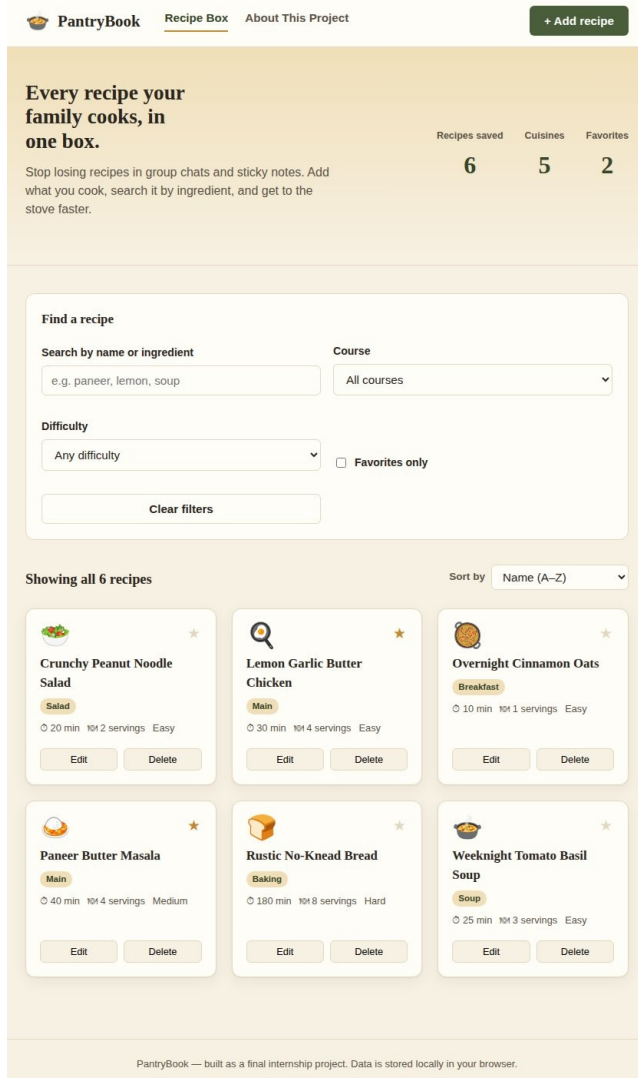
Method * (one step per line)

Rinse the rice until water runs clear.
Heat oil and fry the cumin until fragrant.

Icon

 Curry / rice

Desktop — validated Add/Edit recipe form.



Tablet layout (834px) — filters reflow into a two-column panel.



+ Add recipe

Every recipe your family cooks, in one box.

Stop losing recipes in group chats and sticky notes. Add what you cook, search it by ingredient, and get to the stove faster.

Recipes saved

6

Cuisines

5

Favorites

2

Find a recipe

Search by name or ingredient

e.g. paneer, lemon, soup

Course

All courses

Difficulty

Any difficulty

☐ Favorites only

Clear filters

Showing all 6 recipes

Sort by

Name (A-Z)



Crunchy Peanut Noodle Salad

Salad

🕒 20 min 🍽️ 2 servings Easy

Edit

Delete



Lemon Garlic Butter Chicken

Main

🕒 30 min 🍽️ 4 servings Easy

Edit

Delete

Mobile layout (390px) — single-column, thumb-friendly controls.

GitHub Repository Link

<https://github.com/<your-username>/pantrybook> (update after pushing — see README for exact push commands)

Live Website Link

<https://<your-username>.github.io/pantrybook/> (update after enabling GitHub Pages)

Challenges Faced

- Keeping the filter, search and sort controls working together without re-running redundant DOM renders.
- Designing a card grid that stays legible from a 1440px desktop down to a 375px phone without separate markup.
- Validating a multi-line ingredients/steps textarea without a heavier form library.

Solutions Implemented

- Centralised all list logic in one `getFilteredRecipes()` function that search, filters and sort all feed into, then a single `renderGrid()` call re-renders the DOM — avoiding inconsistent state.
- Used CSS Grid with `auto-fill/minmax` for the card grid and a small set of media query breakpoints (900px, 600px) so the layout reflows rather than being rebuilt per device.
- Split textarea input on newlines, trimmed and filtered empty lines, then checked array length against a minimum before allowing submit, with inline error text per field.

Key Learnings

- How to structure JavaScript around an array of objects as a single source of truth, with the DOM as a pure function of that state.
- Practical Flexbox vs. Grid decisions — Flexbox for one-dimensional rows (header, filters), Grid for the two-dimensional card layout.
- A disciplined Git habit: staging related files together and writing a commit message that describes the change, not just 'update'.

Future Improvements

- Sync recipes to a real backend/database so a collection can be shared across devices.
- Add photo uploads per recipe instead of icon-only cards.
- Add unit conversion and a 'scale servings' control for ingredient quantities.